

1. Current Feature Inventory

Game Tracking & Earnings Management

- **Record Games:** Add games with date, location, game number, transportation cost, food cost, referee payment, payment status, payment date, and observations
- **Game Database:** Stores 56+ games, supports SQLite/MySQL/PostgreSQL or JSON fallback
- **Edit Games:** Modify existing game records
- **Delete Games:** Remove games from the database
- **Search Games:** Search by location, game number with fuzzy matching
- **Sort Games:** Sort by date (primary), amount left, payment status, etc. with click-to-reverse
- **Export Games:** Export all seasons to CSV/XLSX with configurable date range
- **Import Games:** Import CSV/XLSX files with header detection and conflict resolution
- **Mark Paid:** Batch mark games as paid
- **Seasonal Filtering:** View games by season (2025/2026 currently)
- **Summary Stats:** Total earnings, amount left (unpaid), number of games visible in UI

Rulebook Navigation & Search

- **PDF Viewer:** Display rulebook PDF with manual page navigation
- **Embedded Images:** Extract and display images from rulebook pages
- **Rule Sections:** Parse rules into sections (Rule 1-18+) from table of contents
- **Search:** Basic substring search across rule titles and content (case-insensitive)
- **Zoom:** Manual zoom in/out for PDF viewing

Testing & Knowledge Improvement

- **Quiz Modes:** Random, by category, unanswered questions, wrong answers
- **Test Sessions:** Track individual quiz attempts with timestamps and attempt numbers
- **Session History:** Persist all test sessions to JSON with full attempt details
- **Progress Tracking:** Remember answered questions, correct/wrong answers across sessions
- **Question Pool:** 300+ questions with rule references and explanations
- **Test Timer:** Display elapsed time for current session (visual only, not enforced)
- **Statistics:** Calculate overall success rate and stats by rule category
- **Question of the Day:** Daily random question on dashboard with answer checking

Dashboard & Reporting

- **KPI Cards:** Show test attempts, accuracy percentage, games this season
 - **Visual Theme:** Dark blue handball-themed UI with custom colors and fonts
 - **Statistics Page:** Show stats by rule category, success rates, session count
 - **Data Persistence:** All data stored locally (games in DB, tests in JSON, progress in JSON)
-

2. Identified Gaps

Critical Gaps

1. **Rulebook Navigation is Fragmented**
 - PDF viewer requires manual page navigation
 - Search results don't link back to PDF pages
 - No way to jump from questions directly to relevant rules (despite rule references being captured)
 - Rules are parsed but relationships between questions and rules are one-way only
 2. **Game Analytics Completely Missing**
 - 56 games tracked but no analysis of referee performance, earnings trends, or patterns
 - No metrics like "earnings per game", "most frequent location", "average payment delay"
 - No filtering by location, referee role, or other patterns
 - Payment tracking is crude (Yes/No only)
 3. **Test-Game Disconnect**
 - Testing is isolated; no connection to actual game scenarios
 - No way to note "I got this question wrong, and it came up in game X"
 - No way to review a game's relevance to test performance
 4. **Multi-User Readiness: Zero**
 - All data is singleton (one referee's data per installation)
 - No user accounts, logins, or multi-user database design
 - No sharing of test sessions or game records
 - No collaborative features (e.g., comparing performance with other referees)
 5. **Data Export/Sharing Limitations**
 - Games export to CSV works, but tests cannot be exported
 - No PDF generation for reports (e.g., "test report for certification")
 - No way to share progress with coaches or peers
 6. **Test Question References Underutilized**
 - Questions have rule references extracted but not displayed during tests
 - Answer explanations are shown but rule links are not interactive
 - No way to jump from test feedback to the relevant rule
 7. **Payment Workflow Incomplete**
 - Payment date is optional and freeform
 - No alerts for overdue payments
 - No payment recovery tracking (difference between expected and actual payment)
 - No summary of payment delays by season
 8. **Rulebook Content Not Indexed**
 - Images are extracted but not indexed or searchable
 - Rule sections have no metadata (difficulty, frequency in tests, etc.)
 - No way to find which rules are covered in the test pool
-

3. Unfinished Features Found in Code

No explicit TODO comments found, but several **partially built features**:

1. **Question of the Day (QOD)** (submit_question_of_day method)
 - Implemented in dashboard but uses only the first loaded question
 - Never rotates/randomizes per actual day
 - No caching or persistence

2. **Test Timer** (`_tick_test_timer`, `_stop_test_timer`)
 - Displays elapsed time but has no time limit
 - No session timeout enforcement
 - Used only for UI display, not for validation
 3. **Database Multi-Backend Support**
 - MySQL/PostgreSQL support is coded but untested
 - Error messages suggest it's rarely used
 4. **Rulebook PDF Loading**
 - PyMuPDF integration exists but is conditional on optional dependency
 - Falls back silently if PyMuPDF missing
 - No error messaging to user
-

4. Feature Suggestions

Tier 1 — High Impact, Low Effort

1.1 — Bidirectional Rulebook-Question Links

- **What it does:** When viewing a question during a test, show "Related Rules" sidebar with links to jump to those rule sections in the PDF viewer. When viewing a rule, show "Questions about this rule" list.
 - **Why it matters:** Referees studying for certification need to connect rules to test questions. Currently, rule references exist in the data but aren't exposed during study sessions.
 - **What it connects to:** Rulebook viewer + Test questions + Rule references already parsed
 - **How to build it:**
 - Add a new UI panel in the test question display showing rule references
 - Add a click handler that jumps to the rulebook page for that rule section
 - In the rulebook page, query the test dataset to find all questions referencing the current rule
 - Display as a list in a sidebar
 - **Effort estimate:** Small (4–6 hours)
 - **Dependencies:** Requires rulebook dataset loaded (already done)
-

1.2 — Payment Alerts & Overdue Summary

- **What it does:** Add a "Payments Due" section to dashboard showing games where payment is overdue (today's date > paymentDate expected + 30 days) or missing. Color-code by severity.
- **Why it matters:** Referees need to track down late payments. Currently, payment tracking is just a Yes/No toggle with no visibility into delays.
- **What it connects to:** Game tracking + Dashboard
- **How to build it:**
 - Calculate expected payment date as date + 3 days (configurable)
 - Query games where (paid=No AND date + 3 days < today) OR (paid=Yes AND paymentDate empty)

- Display count and list on dashboard; add a "Payment Recovery" button to filter games page
 - **Effort estimate:** Small (3–4 hours)
 - **Dependencies:** None
-

1.3 — Improved Rulebook Search with Page Jumps

- **What it does:** When searching the rulebook, show results as a sortable list. Clicking a result jumps the PDF viewer to that page and highlights the match.
 - **Why it matters:** Current search works but is disconnected from the PDF view—results don't help you navigate to the rule.
 - **What it connects to:** Rulebook viewer + Search functionality
 - **How to build it:**
 - Store page number with each rule section during parsing (already done: `start_page`, `end_page`)
 - When search completes, format results with page numbers
 - Add click handler to jump PDF to that page and scroll to highlighted text
 - **Effort estimate:** Small (3–4 hours)
 - **Dependencies:** PyMuPDF (already installed)
-

1.4 — Rule Difficulty Tagging

- **What it does:** Automatically tag rules based on how frequently they appear in test questions. Display difficulty (Easy, Medium, Hard) in the rulebook sidebar.
 - **Why it matters:** Referees studying should know which rules are tested more frequently—helps prioritize.
 - **What it connects to:** Test questions + Rules
 - **How to build it:**
 - Count how many questions reference each rule
 - Tag as Easy (0-2 Q's), Medium (3-5 Q's), Hard (6+ Q's)
 - Store in rule metadata; display in rulebook UI
 - **Effort estimate:** Small (2–3 hours)
 - **Dependencies:** Rule references already extracted
-

1.5 — Question of the Day (Complete)

- **What it does:** Pick a truly random question each calendar day. Show it on the dashboard with persistence—same question shown to the user all day. Highlight if they've already answered it.
- **Why it matters:** Currently broken (uses first question always). A real daily challenge encourages daily app usage.
- **What it connects to:** Dashboard + Test questions + Progress tracking
- **How to build it:**
 - Use date-based seed (today's date) to pick a reproducible random question
 - Persist user's answer for the day in progress JSON
 - Show "You already answered this" + old answer if applicable

- **Effort estimate:** Small (2–3 hours)
 - **Dependencies:** None
-

● Tier 2 — High Impact, Medium Effort

2.1 — Game Analytics Dashboard

- **What it does:** New analytics page showing earnings trends (chart by month), average payment delay, most frequent locations, win rate by location (if applicable), total transportation/food costs breakdown.
 - **Why it matters:** Referees tracked 56 games but have no visibility into patterns. Analytics reveal which locations/seasons are most profitable, where delays happen, etc.
 - **What it connects to:** Game tracking + Statistics page
 - **How to build it:**
 - Add new page to tab bar: "Analytics"
 - Query game data to aggregate by month, location, season
 - Use matplotlib or similar to render line charts (earnings over time), bar charts (costs by category)
 - Add filters by season/location
 - **Effort estimate:** Medium (1–2 days)
 - **Dependencies:** Requires matplotlib or plotly (not currently in requirements.txt; add it)
-

2.2 — Test Performance Reports & Export

- **What it does:** Generate a PDF or HTML report showing test performance by rule category, weakest areas, improvement over time. Allow export for sharing or archival.
 - **Why it matters:** Referees may need to show progress to coaches/associations. Currently, no way to export tests or generate a summary report.
 - **What it connects to:** Test sessions + Statistics
 - **How to build it:**
 - Query session history and calculate per-category pass rates
 - Use reportlab (PDF) or jinja2 + HTML to render a formatted report
 - Include date range filter, charts of improvement
 - Save to file; optionally email (future feature)
 - **Effort estimate:** Medium (1–2 days)
 - **Dependencies:** reportlab or jinja2 (add to requirements)
-

2.3 — Rulebook Indexing & Metadata

- **What it does:** During dataset build, extract keywords, difficulty, question count, and image count for each rule section. Store as metadata; display in UI.
- **Why it matters:** Rulebook is just text right now. Metadata helps referees quickly gauge which rules are critical and heavily tested.
- **What it connects to:** Rulebook content building
- **How to build it:**

- Enhance `build_handball_dataset()` to compute metadata for each rule (questions referencing it, keywords extracted from content, image count)
 - Store as "metadata" field in rule object
 - Display in rulebook sidebar (e.g., "Rule 1: Basic Rules — Difficulty: Hard (8 questions)")
 - **Effort estimate:** Medium (1 day)
 - **Dependencies:** None (metadata computation is straightforward)
-

2.4 — Test Session Replay / Review

- **What it does:** Allow reviewing past test sessions: see which questions were missed, how long was spent per question, what the explanation was. Highlight areas for improvement.
 - **Why it matters:** Currently, test sessions are recorded but not reviewable. Referees learn more if they can review mistakes.
 - **What it connects to:** Test engine + Session history
 - **How to build it:**
 - Add "Review Past Sessions" button on statistics page
 - Show list of sessions; clicking one opens a "replay" view with each attempt shown side-by-side with the correct answer
 - Highlight errors in red; show time spent per question
 - **Effort estimate:** Medium (1–1.5 days)
 - **Dependencies:** Session data already persisted
-

2.5 — Flexible Payment Status Workflow

- **What it does:** Upgrade payment tracking from Yes/No to a proper workflow: Pending → Claimed → Received (with dates). Add a payment notes field. Summary shows payment age.
 - **Why it matters:** Referees need to track intermediate states (claimed but not yet received). Currently too coarse.
 - **What it connects to:** Game tracking + Dashboard
 - **How to build it:**
 - Migrate schema: add `payment_status_enum` (Pending/Claimed/Received) and `payment_claim_date`
 - Update game dialog to allow status selection and date entry
 - Recalculate "days since claim" in summary
 - Filter games by status
 - **Effort estimate:** Medium (1–1.5 days)
 - **Dependencies:** Database schema migration needed
-

● Tier 3 — Multi-User / Sharing Ready

3.1 — Referee Profiles & Multi-User Support

- **What it does:** Add a user account system. Each referee has a profile (name, league, certification level) and isolated data. Referees can switch profiles or share data with others (TBD).

- **Why it matters:** App will be used by multiple referees (stated in brief). Currently, it's single-user only. This is foundational for everything downstream.
 - **What it connects to:** All data storage, UI, authentication
 - **How to build it:**
 - Add users table to schema (id, username, email, profile_created, certification_level)
 - Migrate all game/test data to be user-scoped (add foreign keys)
 - Add login screen at app startup; store current user in session
 - Add profile editor (name, league, cert level)
 - **Architectural change:** GameDataManager, TestResults, QuizEngine all need user context
 - **Effort estimate:** Large (3–4 days) — involves schema migrations and UI restructuring
 - **Dependencies:** Must be done first; unlocks 3.2, 3.3, 3.4
-

3.2 — Shared Test Benchmarks & Comparisons

- **What it does:** Once multi-user exists, allow referees to compare their test performance anonymously against league/national benchmarks. Show "You're in the top 30% for Rule 5."
 - **Why it matters:** Referees want to know how they stack up. Encourages improvement.
 - **What it connects to:** Multi-user profiles + Test statistics
 - **How to build it:**
 - Collect performance data from all referees (anonymized)
 - Compute percentiles by rule category
 - Display comparison chart on statistics page
 - **Privacy consideration:** Must be opt-in and anonymized
 - **Effort estimate:** Large (2–3 days) — requires aggregation logic and privacy controls
 - **Dependencies:** 3.1 (multi-user)
-

3.3 — Data Sharing / Export for Coaches

- **What it does:** Generate a shareable link or PDF that shows a referee's performance (test results, game summary, earnings). Coaches can review without full app access.
 - **Why it matters:** Coaches need to assess referee readiness. Currently, no way to share data.
 - **What it connects to:** Test reports + Game summaries
 - **How to build it:**
 - Generate a static HTML/PDF with test results and game stats
 - Include a timestamp and optional password protection
 - Store on local filesystem or cloud (future: cloud upload)
 - **Effort estimate:** Medium–Large (2–3 days) — report generation + optional cloud integration
 - **Dependencies:** 3.1 helpful but not strictly required
-

3.4 — League/Tournament Mode

- **What it does:** Support tracking referee assignments for specific tournaments. Log game-level assignments (team A vs. Team B, role, outcome) and enable peer feedback.
- **Why it matters:** Multi-referee workgroups need coordination. Currently, no support for league structures or peer feedback.

- **What it connects to:** Game tracking + Multi-user
 - **How to build it:**
 - Add tournaments table and assignments linking referees to games
 - Add optional "peer feedback" field to games (other refs comment on your calls)
 - New "Tournament View" page showing all referees' assignments
 - **Architectural change:** Game schema needs optional foreign key to tournament
 - **Effort estimate:** Large (3–4 days)
 - **Dependencies:** 3.1 (multi-user) helpful but could work without it
-

3.5 — Certification Path Tracking

- **What it does:** Allow organization admins to define a certification curriculum (e.g., "Pass Rule 1–3 tests with 80%+, pass practice game evaluation"). Track individual progress toward certification.
 - **Why it matters:** Handball associations use certification systems. This app could support them directly.
 - **What it connects to:** Test results + Multi-user
 - **How to build it:**
 - Add certification_requirements table (rule range, min % score, due date)
 - Add certification_progress tracking (status, date completed, score)
 - New admin panel to define requirements
 - Display "Certification Roadmap" on user dashboard
 - **Effort estimate:** Large (4+ days) — requires admin UI and complex logic
 - **Dependencies:** 3.1 (multi-user)
-

5. Recommended Starting Point

Start with 1.1 (Bidirectional Rulebook-Question Links)

Why:

- **Highest immediate user value:** Referees studying are your primary use case. Connecting test questions to rules directly solves the #1 pain point (rulebook navigation + test isolation).
- **Minimal effort:** 4–6 hours, uses existing data structures, no schema changes, no new dependencies.
- **Unlocks downstream work:** Once question-rule linking is visible in the UI, it's natural to build on that (add difficulty tags, better search, etc.).
- **Validates the architecture:** This feature tests whether the refactored codebase is easy to extend—if it is, other features follow quickly.
- **Low risk:** It's purely additive; doesn't break existing features.

Implementation order (after 1.1):

1. **1.2 (Payment Alerts)** — simple, high-value for users
 2. **1.3 (Rulebook Search + Page Jumps)** — improves core workflow
 3. **2.1 (Game Analytics)** — unlocks data-driven insights
 4. **3.1 (Multi-user)** — architectural prerequisite for future sharing/collaboration
-

✅ Feature discovery complete. Review the suggestions above and tell me which features you want to build — I'll help you plan and implement them one by one.